CLASS - XII PROJECT FILE

ACADEMIC YEAR: 2026 - 27

P R O J E C T R E P O R T O N

Hospital Management System
(Python and MySQL)

BOARD ROLL NO	:	.....
NAME	:	.....
CLASS	:	.....
SUBJECT	:	COMPUTER SCIENCE (083)
PROJECT GUIDE	:	.....

TABLE OF CONTENTS

- Certificate
- Acknowledgement
- Preface of Project
- Project Description
- Hardware & Software Specifications
- Python Modules Used
- Database Design
- Project Code
- Sample Outputs
- Scope of the Project
- Bibliography

• • • • • • • • • • •

GREEN VALLEY
SCHOOL FOR CHILDREN

Certificate

This is to certify that .....

Board Roll No: ..... has successfully completed the Project Work entitled Hospital Management System in the subject COMPUTER SCIENCE (083) laid down for the purpose of Practical Examination in Class XII for the Academic Session: 2026 - 27.

_____	_____
Internal Examiner's Signature	**External Examiner's Signature**
_____	_____
Principal's Signature	**School's Stamp**

• • • • • • • • • • •

ACKNOWLEDGEMENT

Apart from one's own effort, the success of any project depends largely on the encouragement and guidance of many others. I take this opportunity to express my gratitude to the people who supported the completion of this project.

I express my heartfelt gratitude to my parents for their constant encouragement while carrying out the project.

I gratefully acknowledge the contribution of everyone who helped shape this work and encouraged me to understand how information systems can support hospital administration.

I express my deep sense of gratitude to the Principal, ..., who has continuously motivated and supported us.

My sincere thanks to ....., Project Guide, for reviewing the work and helping me resolve the problems that occurred during its implementation.

The guidance and support received from all the members who contributed to this project were vital to its completion. I am grateful for their constant support and help.

• • • • • • • • • • •

PREFACE OF PROJECT

This project, a Hospital Management System, was developed as a practical introduction to organizing common hospital records. It combines programming and database concepts to demonstrate how patient, doctor, appointment, admission, and billing information can be stored and retrieved.

The project uses Python for application logic and MySQL for persistent data storage. The Python console menu provides a simple way to register patients, review doctors, schedule appointments, update appointment status, and record basic charges.

Throughout the project, students can explore database connectivity, create, read, update, and delete operations, input validation, foreign keys, indexes, and transaction handling. Each part demonstrates how an application can interact with a relational database.

The primary focus is educational. The database structure and console application are designed to be understandable and adaptable for learning. The sample records are fictional and should not be replaced with real patient information.

The project also introduces data integrity and error handling. It is a foundation for further study and is not a complete clinical system or a substitute for production security, privacy, and compliance review.

• • • • • • • • • • •

PROJECT DESCRIPTION

Title: Hospital Management System

Introduction:

The Hospital Management System is an educational software application for organizing selected administrative records in a small hospital setting. It is developed with Python for application processing and MySQL for relational database management. The system provides a simple console interface for commonly demonstrated record-keeping tasks.

Objective:

The main objective is to create a clear and practical system that can register patients, maintain a doctor directory, schedule appointments, track appointment outcomes, and prepare basic bills. Structured tables, validation, and database constraints help reduce inconsistent data entry.

Features and Functionalities:

- Patient Registration: saves medical record number, name, date of birth, contact details, and emergency contact information.
- Doctor Directory: stores provider details and links each doctor to a department.
- Appointment Scheduling: records date, time, patient, doctor, reason, and status; checks for overlapping appointments.
- Admission and Bed Records: stores ward and bed information and prevents an occupied bed from being assigned twice.
- Billing: records bill headers and itemized charges for a patient.
- Data Validation and Error Handling: checks required fields and reports invalid inputs and database errors.

PROJECT DESCRIPTION

Technology Stack:

> Programming Language: Python 3

> Database Management System: MySQL 8.0.16 or newer

> Database Connector: mysql-connector-python

> Development Environment: Python IDLE or any Python IDE; MySQL Server and a terminal/SQL client

Target Audience:

This project is primarily intended for students and educators in computer science. It demonstrates how an application can use a relational database to support routine record management.

Conclusion:

The Hospital Management System brings together programming and database topics in a practical example. It helps students understand how a set of related records can be designed, queried, and updated while maintaining basic data integrity.

Important Scope Note:

This report and its accompanying program are an educational demonstration only. They do not provide clinical decision support, role-based access, audit logging, encrypted backups, or production-grade privacy controls. Do not use this sample with real patient data.

HARDWARE & SOFTWARE SPECIFICATIONS

HARDWARE SPECIFICATIONS

Processor : Intel Core i3 and above

Hard Disk : 100 GB or above

RAM : 4 GB or above

Display : 1366 x 768 or higher

SOFTWARE SPECIFICATIONS

Operating System : Windows 10 or above

Platform : Python 3.x / IDLE or IDE

Database : MySQL Server 8.0.16 or above

Connector : mysql-connector-python 9.x

Languages : Python and SQL

Note: Configure a local MySQL Server service before running the application. The app reads its connection values from HMS_DB_HOST, HMS_DB_PORT, HMS_DB_USER, HMS_DB_PASSWORD, and HMS_DB_NAME. Keep the password private.

• • • • • • • • • • •

PYTHON MODULES USED

1. mysql.connector

Used to connect Python to MySQL and execute parameterized SQL statements.

2. os

Reads database connection settings from environment variables.

3. datetime

Validates dates and calculates appointment end times.

4. decimal

Handles bill quantities and prices as decimal values.

5. mysql.connector.Error

Catches database errors and displays a useful message.

• • • • • • • • • • •

DATABASE DESIGN



MySQL relational design - primary and foreign keys

```
departments   1 ---- many  doctors
patients      1 ---- many  appointments  many ---- 1 doctors
patients      1 ---- many  admissions    many ---- 1 beds
patients      1 ---- many  bills         1 ---- many bill_items

All tables use InnoDB; foreign keys preserve relationships.
Appointment and admission dates are checked for validity.
```

PROJECT CODE

```
import os
import mysql.connector
from mysql.connector import Error


def connect_db():
    return mysql.connector.connect(
        host=os.getenv("HMS_DB_HOST", "127.0.0.1"),
        port=int(os.getenv("HMS_DB_PORT", "3306")),
        user=os.getenv("HMS_DB_USER", "root"),
        password=os.getenv("HMS_DB_PASSWORD", ""),
        database=os.getenv("HMS_DB_NAME", "hospital_management"),
    )


def add_patient(mrn, first, last, dob, sex, phone=None, email=None):
    sql = """INSERT INTO patients
        (medical_record_no, first_name, last_name, date_of_birth,
         sex, phone, email)
        VALUES (%s, %s, %s, %s, %s, %s, %s)"""
    db = connect_db()
    try:
        cur = db.cursor()
        cur.execute(sql, (mrn, first, last, dob, sex, phone, email))
        db.commit()
        return cur.lastrowid
    except Error:
        db.rollback()
        raise
    finally:
        db.close()


def list_patients():
    with connect_db() as db:
        cur = db.cursor()
        cur.execute("""SELECT patient_id, medical_record_no,
            first_name, last_name, date_of_birth
            FROM patients ORDER BY last_name, first_name""")
        return cur.fetchall()
```

• • • • • • • • • • •

PROJECT CODE

```
from datetime import timedelta


def book_appointment(patient_id, doctor_id, starts_at, reason=None):
    ends_at = starts_at + timedelta(minutes=30)
    db = connect_db()
    try:
        db.start_transaction()
        cur = db.cursor()
        # Serialize bookings for the same patient and doctor.
        cur.execute("SELECT patient_id FROM patients WHERE patient_id=%s FOR UPDATE",
                    (patient_id,))
        if not cur.fetchone():
            raise ValueError("Patient not found")
        cur.execute("SELECT doctor_id FROM doctors WHERE doctor_id=%s AND active=TRUE FOR UPDATE",
                    (doctor_id,))
        if not cur.fetchone():
            raise ValueError("Active doctor not found")
        cur.execute("""SELECT 1 FROM appointments WHERE status='scheduled'
            AND starts_at < %s AND ends_at > %s
            AND (doctor_id=%s OR patient_id=%s) LIMIT 1""",
            (ends_at, starts_at, doctor_id, patient_id))
        if cur.fetchone():
            raise ValueError("Time slot is already occupied")
        cur.execute("""INSERT INTO appointments
            (patient_id, doctor_id, starts_at, ends_at, reason)
            VALUES (%s, %s, %s, %s, %s)""",
            (patient_id, doctor_id, starts_at, ends_at, reason))
        db.commit()
        return cur.lastrowid
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()


def create_bill(patient_id, description, quantity, unit_price):
    db = connect_db()
    try:
        cur = db.cursor()
        cur.execute("INSERT INTO bills (patient_id) VALUES (%s)",
                    (patient_id,))
        bill_id = cur.lastrowid
        cur.execute("""INSERT INTO bill_items
            (bill_id, description, quantity, unit_price)
            VALUES (%s, %s, %s, %s)""",
            (bill_id, description, quantity, unit_price))
        db.commit()
        return bill_id
    except Error:
        db.rollback()
        raise
    finally:
        db.close()
```

• • • • • • • • • • •

PROJECT CODE

```
def main():
    actions = {
        "1": ("List patients", show_patients),
        "2": ("Add patient", add_patient),
        "3": ("List doctors", show_doctors),
        "4": ("Book appointment", book_appointment),
        "5": ("List appointments", show_appointments),
        "6": ("Update appointment status", update_appointment_status),
        "7": ("List bills", show_bills),
        "8": ("Create bill", create_bill),
    }
    while True:
        print("\nHOSPITAL MANAGEMENT SYSTEM")
        for key, (label, _) in actions.items():
            print(f"{key}. {label}")
        print("0. Exit")
        choice = input("Choose an option: ").strip()
        if choice == "0":
            print("Goodbye.")
            return
        action = actions.get(choice)
        if action is None:
            print("Invalid option.")
            continue
        try:
            action[1]()
        except Error as exc:
            print(f"Database error: {exc}")


if __name__ == "__main__":
    main()
```

SQL statements use placeholders (%s) so user input is not
joined into query strings. Each write is committed only after
validation; failed transactions are rolled back.

• • • • • • • • • • •

PROJECT OUTPUTS

Main Menu

```
HOSPITAL MANAGEMENT SYSTEM
1. List patients
2. Add patient
3. List doctors
4. Book appointment
5. List appointments
6. Update appointment status
7. List bills
8. Create bill
0. Exit
```

Add Patient

```
Medical record number: DEMO-0001
First name: Sample
Last name: Patient
Date of birth (YYYY-MM-DD): 2000-01-15
Patient added (ID 1).
```

Book Appointment

```
Patient ID: 1
Doctor ID: 1
Start date/time (YYYY-MM-DD HH:MM): 2026-11-02 10:00
Reason for visit: General consultation
Appointment booked (ID 1) for 2026-11-02 10:00.
```

• • • • • • • • • • •

PROJECT OUTPUTS

View Doctors

```
ID | Doctor | Specialty | Department | Phone
1 | Dr. Asha Menon | General Physician | General Medicine | -
2 | Dr. Ravi Shah | Cardiologist | Cardiology | -
```

Create Bill

```
Patient ID: 1
Charge description: Consultation
Quantity: 1
Unit price: 500.00
Bill 1 created; total 500.00.
```

Update Appointment Status

```
Appointment ID: 1
New status (completed/cancelled/no_show): completed
Appointment status updated.
```

• • • • • • • • • • •

SCOPE OF THE PROJECT

The scope of the Hospital Management System project includes the following key areas:

1. Database Management:

Stores patient, doctor, department, appointment, admission, bed, bill, and bill item records with primary keys, foreign keys, uniqueness rules, and validation constraints.

2. Patient and Provider Records:

Supports adding and listing patient records and maintains a directory of active doctors grouped by department.

3. Appointment Management:

Books 30-minute appointments, checks for overlapping scheduled appointments, and updates a booking to completed, cancelled, or no-show.

4. Admissions and Beds:

The schema stores admission history and bed assignments. A unique active-bed constraint prevents two active admissions from occupying the same bed.

5. Billing:

Stores bill headers and itemized charges. Totals are calculated from quantity multiplied by unit price.

6. User Interface and Error Handling:

A text menu guides the user; input validation and database messages help explain common problems.

Limitations: This educational prototype has no login roles, audit history, real-time clinical workflows, backups, or regulatory certification. It must not be used with real patient data.

• • • • • • • • • • •

BIBLIOGRAPHY



I took the information for this project from the following sources:

Reference Books:

1. Computer Science by Sumita Arora.

2. Complete Reference with Python.

3. NCERT Computer Science textbook.

Websites:

https://docs.python.org/3/

https://dev.mysql.com/doc/refman/8.4/en/

https://dev.mysql.com/doc/connector-python/en/